

Buổi 6 — Quan hệ dữ liệu, Include, tìm kiếm và phân trang

Môn: Lập trình Web

Thời lượng: 3 giờ

Giảng viên: _____

Mục tiêu buổi học

Kết thúc buổi học, sinh viên có thể:

1. Thiết kế quan hệ **một-nhiều** và **nhiều-nhiều** bằng EF Core.
2. Dùng **navigation property** và **Include** để nạp dữ liệu liên kết.
3. Viết truy vấn LINQ để **tìm kiếm** và **sắp xếp**.
4. Cài đặt **phân trang** cho danh sách bài viết.

Lộ trình buổi học

Phần	Nội dung	Thời lượng
1	Quan hệ một-nhiều, nhiều-nhiều	30 phút
2	<code>Include</code> — nạp dữ liệu liên kết	20 phút
3	Tìm kiếm + sắp xếp bằng LINQ	25 phút
4	Phân trang (<code>Skip</code> / <code>Take</code>)	25 phút
5	Live coding Category + tìm kiếm + phân trang	40 phút
6	Thực hành + Git + tổng kết	30 phút

Ôn tập nhanh buổi 5

Trả lời nhanh:

1. Truy vấn EF Core lấy danh sách dùng phương thức async nào?
2. Vì sao cần validation cả server lẫn client?
3. CRUD **Post** của bạn đã lưu bền vững chưa?

Vấn đề mới: dữ liệu thực tế luôn **có liên kết** và số lượng **lớn dần**.

Bối cảnh: vì sao tách bảng?

Không tốt: lưu tên chuyên mục trong mỗi bài

Posts: | Id | Title | CategoryName |

| 1 | A | "Lập trình" |

| 2 | B | "Lập trình" | ← trùng lặp, đổi tên phải sửa nhiều dòng

Chuẩn hóa: tách bảng, liên kết bằng khóa ngoại

Categories: | Id | Name |

| 1 | Lập trình |

Posts: | Id | Title | CategoryId |

| 1 | A | 1 |

| 2 | B | 1 |

Tách bảng → hết trùng lặp, đổi tên chuyên mục chỉ sửa **một** dòng.

Quan hệ một-nhiều

Một `Category` có nhiều `Post` ; mỗi `Post` thuộc một `Category` .

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public List<Post> Posts { get; set; } = new(); // navigation
}

public class Post
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;

    public int CategoryId { get; set; } // khóa ngoại
    public Category? Category { get; set; } // navigation
}
```

EF Core tự nhận quan hệ nhờ quy ước `CategoryId` + `Category` .

Quan hệ nhiều-nhiều

Một `Post` có nhiều `Tag`, một `Tag` gắn nhiều `Post`.

Từ EF Core 5+: chỉ cần navigation ở **hai phía**, EF Core tự tạo bảng nối.

```
public class Tag
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public List<Post> Posts { get; set; } = new();
}

public class Post
{
    // ...
    public List<Tag> Tags { get; set; } = new();
}
```

EF Core sinh bảng nối `PostTag` (`PostsId`, `TagsId`) tự động.

Cập nhật DbContext + migration

```
public DbSet<Post> Posts => Set<Post>();  
public DbSet<Category> Categories => Set<Category>();  
public DbSet<Tag> Tags => Set<Tag>();
```

```
dotnet ef migrations add AddCategoryAndTag  
dotnet ef database update
```

Quy trình quen thuộc: đổi Entity → migration → database update .

Include — nạp dữ liệu liên kết

⚠ Mặc định EF Core **không** tự nạp navigation → `post.Category` là `null`.

```
var posts = await _context.Posts
    .Include(p => p.Category)    // nạp chuyên mục
    .Include(p => p.Tags)        // nạp danh sách thẻ
    .ToListAsync();
```

- Không `Include` → `post.Category == null` → hiển thị rỗng/lỗi
- Có `Include` → nạp kèm dữ liệu liên kết
- Quan hệ sâu hơn → dùng `ThenInclude`

Tìm kiếm + sắp xếp bằng LINQ

```
public async Task<IActionResult> Index(string? search, string? sort)
{
    var query = _context.Posts.Include(p => p.Category).AsQueryable();

    if (!string.IsNullOrEmpty(search))
        query = query.Where(p => p.Title.Contains(search));

    query = sort switch
    {
        "title" => query.OrderBy(p => p.Title),
        "oldest" => query.OrderBy(p => p.PublishedAt),
        _ => query.OrderByDescending(p => p.PublishedAt)
    };

    return View(await query.ToListAsync());
}
```

`AsQueryable` nối điều kiện dần; `Contains` → SQL `LIKE`; chỉ chạy khi `ToListAsync`.

Form tìm kiếm

```
<form asp-action="Index" method="get" class="row g-2 mb-3">
  <div class="col-auto">
    <input name="search" value="@Context.Request.Query["search"]"
      class="form-control" placeholder="Tìm theo tiêu đề" />
  </div>
  <div class="col-auto">
    <button type="submit" class="btn btn-primary">Tìm</button>
  </div>
</form>
```

`method="get"` → điều kiện tìm kiếm nằm trên URL, dễ chia sẻ và ghép với phân trang.

Phân trang — Skip & Take

```
int pageSize = 5;
int page = pageNumber ?? 1;

int totalItems = await query.CountAsync();
int totalPages = (int)Math.Ceiling(totalItems / (double)pageSize);

var posts = await query
    .Skip((page - 1) * pageSize) // bỏ qua các trang trước
    .Take(pageSize)             // lấy đúng 1 trang
    .ToListAsync();
```

Trang 1: Skip 0, Take 5 → bản ghi 1–5
Trang 2: Skip 5, Take 5 → bản ghi 6–10
Trang 3: Skip 10, Take 5 → bản ghi 11–15

⚠ Ép kiểu `double` khi chia, nếu không `totalPages` sẽ sai.

ViewModel phân trang

Gom dữ liệu View gọn gàng thay cho nhiều ViewBag :

```
namespace BlogManager.ViewModels;

public class PostListViewModel
{
    public List<Post> Posts { get; set; } = new();
    public int CurrentPage { get; set; }
    public int TotalPages { get; set; }
    public string? Search { get; set; }
    public string? Sort { get; set; }
}
```

Nhớ ViewModel từ Buổi 5 — đây là chỗ nó phát huy tác dụng rõ nhất.

Thanh phân trang trong View

```
<ul class="pagination">
@for (int i = 1; i <= Model.TotalPages; i++)
{
    <li class="page-item @(i == Model.CurrentPage ? "active" : "")">
        <a class="page-link"
            asp-action="Index"
            asp-route-search="@Model.Search"
            asp-route-sort="@Model.Sort"
            asp-route-pageNumber="@i">@i</a>
    </li>
}
</ul>
```

Giữ `search` + `sort` trong link trang → tìm kiếm và phân trang **hoạt động cùng nhau**.

Live coding — Bước 1–2: Category + migration

Tạo `Category`, thêm khóa ngoại vào `Post`, thêm `DbSet<Category>`, rồi:

```
dotnet ef migrations add AddCategoryRelation  
dotnet ef database update
```

Kiểm tra DB Browser: `Posts` có cột `CategoryId`; có bảng `Categories`.

Bước 3: Dropdown chọn chuyên mục

Controller nạp danh sách chuyên mục:

```
public IActionResult Create()
{
    ViewBag.Categories = _context.Categories.ToList();
    return View();
}
```

Form Create/Edit:

```
<select asp-for="CategoryId"
        asp-items="@ (new SelectList(ViewBag.Categories, "Id", "Name"))"
        class="form-select">
    <option value="">-- Chọn chuyên mục --</option>
</select>
```

Bước 4–5: Include + tìm kiếm + phân trang

Danh sách nạp kèm chuyên mục:

```
var posts = await _context.Posts.Include(p => p.Category).ToListAsync();
```

View thêm cột chuyên mục:

```
<td>@post.Category?.Name</td>
```

Cập nhật Action `Index` nhận `search`, `sort`, `pageNumber` → trả `PostListViewModel` ;
thêm form tìm kiếm + thanh phân trang vào View.

Bước 6: Kiểm tra & commit

Tạo > 5 bài viết, rồi thử:

- ☐ Danh sách hiển thị đúng **tên chuyên mục**
- ☐ Tìm theo tiêu đề → lọc đúng
- ☐ Đổi cách sắp xếp → thứ tự thay đổi
- ☐ Mỗi trang tối đa 5 bài; chuyển trang **giữ nguyên** điều kiện tìm kiếm

```
git add .  
git commit -m "Add category relation, search, sort and pagination"
```

Lỗi thường gặp (1/2)

Hiện tượng	Nguyên nhân	Cách xử lý
<code>post.Category</code> là <code>null</code>	Quên <code>Include(p => p.Category)</code>	Thêm <code>Include</code> vào truy vấn
Lỗi khóa ngoại khi lưu	<code>CategoryId</code> không tồn tại / bỏ trống	Kiểm tra dropdown; cân nhắc cho null
Phân trang sai số trang	Chia số nguyên khi tính <code>totalPages</code>	Ép <code>double</code> rồi <code>Math.Ceiling</code>
Tìm kiếm mất khi chuyển trang	Link trang không giữ <code>search / sort</code>	Thêm <code>asp-route-search</code> , <code>asp-route-sort</code>

Lỗi thường gặp (2/2)

Hiện tượng	Nguyên nhân	Cách xử lý
Nhiều-nhiều không tạo bảng nối	Thiếu navigation một phía	Khai báo <code>List<T></code> ở cả hai Entity
<code>Contains</code> hoa/thường bất ngờ	Khác biệt collation	Chấp nhận theo SQLite / chuẩn hóa chuỗi
Trang trắng khi <code>pageNumber</code> quá lớn	Không kẹp trang hợp lệ	Giới hạn <code>page</code> trong <code>[1, totalPages]</code>

Thực hành trên lớp — 25 phút

Đề bài: Hoàn thiện danh sách `/Posts` có liên kết và duyệt dữ liệu:

1. Quan hệ một-nhiều `Category` ↔ `Post` ; hiển thị chuyên mục ở danh sách + chi tiết
2. Chọn chuyên mục khi tạo/sửa bài
3. Tìm kiếm theo tiêu đề + sắp xếp theo ngày (mới/cũ nhất)
4. Phân trang tối đa 5 bài/trang, giữ điều kiện tìm khi chuyển trang

Đầu ra: ảnh chụp danh sách có tìm kiếm + thanh phân trang + commit.

Checkpoint — Kiểm tra hiểu bài

1. Vì sao nên tách chuyên mục ra bảng riêng?
2. `Include` giải quyết vấn đề gì? Quên `Include` thì sao?
3. `Skip` và `Take` phối hợp thế nào để lấy một trang?
4. Vì sao truy vấn LINQ chỉ chạy khi gọi `ToListAsync` ?

Tóm tắt

1. **Một-nhiều**: khóa ngoại (`CategoryId`) + navigation property.
2. **Nhiều-nhiều**: navigation `List<T>` ở hai phía; EF Core tạo bảng nối.
3. EF Core không tự nạp navigation → dùng **`Include`** / **`ThenInclude`** .
4. **Tìm kiếm/sắp xếp** xây dựng dần trên `IQueryable` , chạy khi `ToListAsync` .
5. **Phân trang** dùng `Skip` / `Take` ; tổng trang từ `CountAsync` .
6. Giữ `search` / `sort` trong link trang để mọi chức năng phối hợp.

Bài tập về nhà

Bắt buộc: Danh sách `/Posts` có liên kết — quan hệ Category (hiển thị + chọn khi tạo/sửa), tìm kiếm theo tiêu đề, sắp xếp theo ngày, phân trang 5 bài/trang giữ điều kiện tìm.

Tiêu chí: các chức năng hoạt động, hiển thị chuyên mục, tìm kiếm + phân trang phối hợp, có commit, nộp ảnh chụp danh sách.

Mở rộng: Thêm quan hệ nhiều-nhiều `Post` ↔ `Tag` ; hiển thị thẻ ở trang chi tiết; lọc danh sách theo một thẻ.

Tự kiểm tra: Trả lời 4 câu hỏi ở slide Checkpoint.

Hạn nộp: trước buổi 7 — nộp ảnh chụp và link repository.

Tài nguyên & chuẩn bị buổi sau

Tài liệu tham khảo

- EF Core — Relationships, Loading Related Data (`Include`), many-to-many — Microsoft Learn
- LINQ — `Where` , `OrderBy` , `Skip` , `Take`
- Bootstrap — Pagination

Mốc mã nguồn: `06-starter` · `06-completed`

Buổi 7 — ASP.NET Core Identity

Đăng ký, đăng nhập, đăng xuất, hash mật khẩu; Authorization và Roles.

→ Chuẩn bị: danh sách có quan hệ, tìm kiếm và phân trang chạy được.